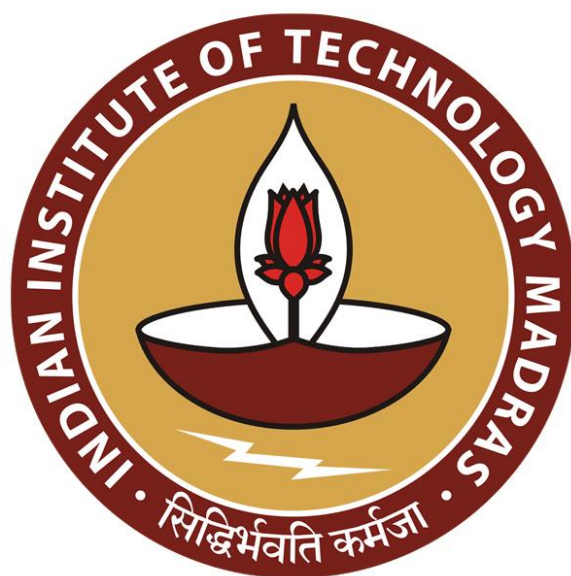




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Monitors in Java

(Refer Slide Time: 0:14)



Monitors in Java

Madhavan Mukund

<https://www.cmi.ac.in/~madhavan>

Programming Concepts using Java

Week 11



So, the last week we looked at concurrent programming in programming languages in general. So, we concluded with the discussion of monitors. So, let us see how monitors work in Java.

(Refer Slide Time: 0:25)



Monitors

- Monitor is like a class in an OO language
 - Data definition — to which access is restricted across threads
 - Collections of functions operating on this data — all are implicitly mutually exclusive
- Monitor guarantees mutual exclusion — if one function is active, any other function will have to wait for it to finish
- Implicit queue associated with each monitor
 - Contains all processes waiting for access

```
monitor bank_account{
    double accounts[100];

    boolean transfer (double amount,
                      int source,
                      int target){
        if (accounts[source] < amount){
            return false;
        }
        accounts[source] -= amount;
        accounts[target] += amount;
        return true;
    }

    double audit(){
        // compute balance across all accounts
        double balance = 0.00;
        for (int i = 0; i < 100; i++){
            balance += accounts[i];
        }
        return balance;
    }
}
```

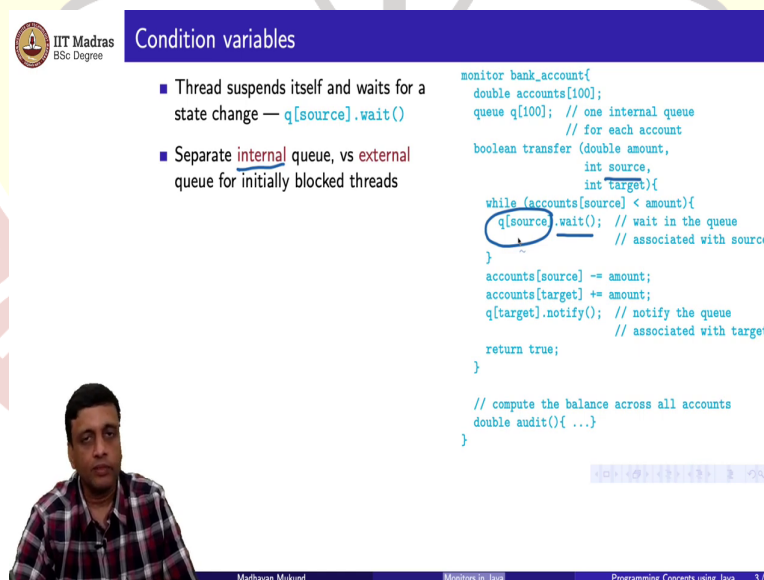


So, remember that a monitor is something like a class. So, it consists of a data definition along with a collection of functions that operate on the data. So, the idea is that these functions for example, here, I have this data, which shows his bank accounts, and I have these two functions transfer and audit, which we wanted to run on these bank accounts. And by putting them into a monitor, by combining them in a monitor, what we say is that these two functions should actually occur in a mutually exclusive fashion.

Or alternatively, one could say that if one function starts to execute, then it must complete execution before any other function in the monitor works. So, they are atomically executed that is without any interruption. As a result, if one of these functions is already operating, so supposing one somebody is already invoked a transfer on this bank account, and somebody else tries to another transfer or an audit, then that second call must wait.

So, there is a queue there is an implicit queue associated with each monitor, which have all the processes which are waiting for access, all the threads which are waiting to access this shared object through these kind of restricted functions, which must execute in a mutually exclusive fashion.

(Refer Slide Time: 1:42)



Condition variables

- Thread suspends itself and waits for a state change — `q[source].wait()`
- Separate **internal** queue, vs **external** queue for initially blocked threads

```
monitor bank_account{
    double accounts[100];
    queue q[100]; // one internal queue
                // for each account
    boolean transfer (double amount,
                     int source,
                     int target){
        while (accounts[source] < amount){
            q[source].wait(); // wait in the queue
                               // associated with source
        }
        accounts[source] -= amount;
        accounts[target] += amount;
        q[target].notify(); // notify the queue
                           // associated with target
        return true;
    }

    // compute the balance across all accounts
    double audit(){ ...}
}
```

Madhavan Mukund Monitors in Java Programming Concepts using Java 3 / 9

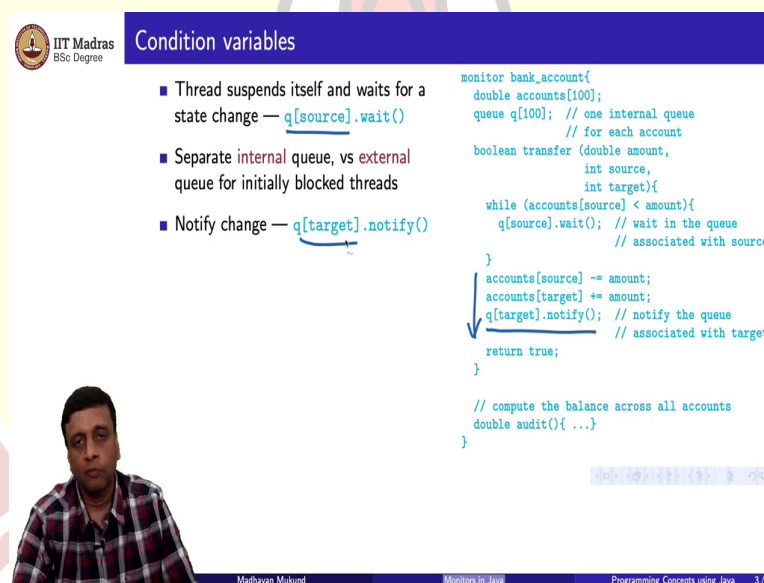
So, one of the things we said about these monitors is that it may be that a function cannot complete because the state of the monitor is not good. So, for instance, when we do a transfer, we would like the source account to have enough funds. But it could be that there is another transfer, which is going to transfer money into this account at which time funds will become

available. So, we would like to suspend this thread and wait for the state of the monitor to change.

Now notice that because the monitor is a synchronized object, this thread must vacate itself from the monitor because otherwise no other thread can come in and update the state. So, we had this wait command. And in particular, we said that this wait is for a very specific condition, it is asking for the source balance to change. So, it will look at this q source, and then it will wait in that source. So, basically, for each account number, I have a waiting queue, which is an internal queue. This is different from the external queue.

So, this is where processes are waiting for some state to change. So, we are waiting for the account value of this particular account to change. So, we have a 100 accounts. So, we have a 100 different places where threads can potentially wait for something interesting to happen.

(Refer Slide Time: 3:00)



Condition variables

- Thread suspends itself and waits for a state change — `q[source].wait()`
- Separate **internal** queue, vs **external** queue for initially blocked threads
- Notify change — `q[target].notify()`

```
monitor bank_account{
    double accounts[100];
    queue q[100]; // one internal queue
                // for each account
    boolean transfer (double amount,
                     int source,
                     int target){
        while (accounts[source] < amount){
            q[source].wait(); // wait in the queue
                               // associated with source
        }
        accounts[source] -= amount;
        accounts[target] += amount;
        q[target].notify(); // notify the queue
                           // associated with target
        return true;
    }

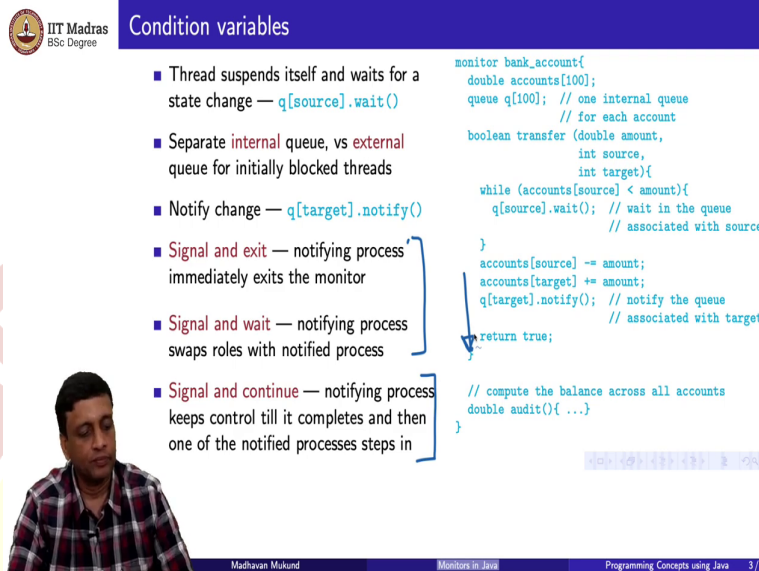
    // compute the balance across all accounts
    double audit() { ... }
}
```

Madhavan Mukund | Monitors in Java | Programming Concepts using Java | 3/9

And symmetrically, what happens is at the end of a successful transfer, is that you have put money into a target account. So, presumably somebody was waiting for money to flow into that account, you can notify them, so we have a wait on one side, which says suspend this thread until something useful happens. And we have a notify, which says notify people who are waiting for this something to happen and that notify and wait can be targeted at specific condition variables.

So, they can be very specialized. So, we are not just saying that any, if any account gets a transfer in then I will try to wake them up only if the account on which they are waiting.

(Refer Slide Time: 3:37)



Condition variables

- Thread suspends itself and waits for a state change — `q[source].wait()`
- Separate **internal** queue, vs **external** queue for initially blocked threads
- Notify change — `q[target].notify()`
- Signal and exit** — notifying process immediately exits the monitor
- Signal and wait** — notifying process swaps roles with notified process
- Signal and continue** — notifying process keeps control till it completes and then one of the notified processes steps in

```
monitor bank_account{
    double accounts[100];
    queue q[100]; // one internal queue
                // for each account
    boolean transfer (double amount,
                     int source,
                     int target){
        while (accounts[source] < amount){
            q[source].wait(); // wait in the queue
                               // associated with source
        }
        accounts[source] -= amount;
        accounts[target] += amount;
        q[target].notify(); // notify the queue
                           // associated with target
    }
    return true;
}

// compute the balance across all accounts
double audit() { ... }
```

Madhavan Mukund | Monitors in Java | Programming Concepts using Java | 3/9

So, when this happens when this wakeup happens, you can do many things. So, for instance, the one that wakes up can immediately quit this is called signal and exit. Or you can have signal and wait, which is that you actually exchange so the process that notifies will swap roles with the process that is waiting, so the one that is waiting will become active and the one that is notified will go back and become inactive. Now these are both in general, very rare, even though in principle, they were allowed with monitors.

And what we will see is that Java actually uses this which is the more logical thing, which is you notify a bunch of processes saying please get ready because something has happened. And maybe one of you will not get to execute but meanwhile you finish this code and exit. So, the process that notifies is allowed to terminate before the other processes get a chance because they have been woken up due to notify.

(Refer Slide Time: 4:31)



Monitors in Java

- Monitors incorporated within existing class definitions
- Function declared `synchronized` is to be executed atomically

```
public class bank_account{
    double accounts[100];

    public synchronized boolean
    transfer(double amount, int source, int target){
        while (accounts[source] < amount){ wait(); }
        accounts[source] -= amount;
        accounts[target] += amount;
        notifyAll();
        return true;
    }

    public synchronized double audit(){
        double balance = 0.0;
        for (int i = 0; i < 100; i++){
            balance += accounts[i];
        }
        return balance;
    }

    public double current_balance(int i){
        return accounts[i]; // not synchronized!
    }
}
```



Madhavan Mukund

Monitors in Java

Programming Concepts using Java

4/9

So, with this background, let us look at how monitors work in Java. So, the nice thing is as we said a monitor is like a class except that in a monitor every method or function that is defined in that class like structure is implicitly a something which has to be executed in a mutually exclusive fashion. So, in Java, we do not have any separate structure for monitors, so, every class can behave like a monitor. So, we just have a class. So, this is the bank account example again.

So, I have this collection of bank accounts. And now the way that we signify in Java that things have to be done carefully is to use this synchronized modifier. So, when we declare a method to be synchronized, what it tells Java is that if this method is invoked by a thread, then do not allow any other method to invoke this method in parallel, but also any other synchronized method. So, all the synchronized methods are implicitly connected by mutually exclusive access to this object.

(Refer Slide Time: 5:44)



Monitors in Java

- Monitors incorporated within existing class definitions
- Function declared `synchronized` is to be executed atomically
- Each object has a lock
 - To execute a `synchronized` method, thread must acquire lock
 - Thread gives up lock when the method exits
 - Only one thread can have the lock at any time

```
public class bank_account{
    double accounts[100];

    public synchronized boolean
    transfer(double amount, int source, int target){
        while (accounts[source] < amount){ wait(); }
        accounts[source] -= amount;
        accounts[target] += amount;
        notifyAll();
        return true;
    }

    public synchronized double audit(){
        double balance = 0.0;
        for (int i = 0; i < 100; i++){
            balance += accounts[i];
        }
        return balance;
    }

    public double current_balance(int i){
        return accounts[i]; // not synchronized!
    }
}
```



Madhavan Mukund

Monitors in Java

Programming Concepts using Java

4/9

So, the way we can think about it is that every object has a lock. So, there is a kind of exclusive access to this object, which is grabbed whenever you do something which is synchronized. So, to execute a synchronized method, you must acquire this lock. And only one process or only one thread can acquire this lock at a time. So, everybody else has to wait. So, there is a queue of waiting processes, those other processes or threads, which need access to this particular object, but which cannot, because some synchronized method is already in progress.

Now, when that synchronous method that is in progress exits, that thread gives up the lock, and now the lock becomes available. And one of the other threads that is waiting is scheduled. So, the important thing is that we are guaranteed that there is with every object, it is not with the class, it is with the object. So, every instance of bank account has got a lock. So, the methods that operate on that instance, on that object, which are synchronized are guaranteed to be mutually exclusive, because only one of them can have the lock at a time.

(Refer Slide Time: 6:51)



Monitors in Java

- Monitors incorporated within existing class definitions
- Function declared `synchronized` is to be executed atomically
- Each object has a **lock**
 - To execute a `synchronized` method, thread must acquire lock
 - Thread gives up lock when the method exits
 - Only one thread can have the lock at any time
- Wait for lock in external queue

```
public class bank_account{
    double accounts[100];

    public synchronized boolean
    transfer(double amount, int source, int target){
        while (accounts[source] < amount){ wait(); }
        accounts[source] -= amount;
        accounts[target] += amount;
        notifyAll();
        return true;
    }

    public synchronized double audit(){
        double balance = 0.0;
        for (int i = 0; i < 100; i++)
            balance += accounts[i];
        return balance;
    }

    public double current_balance(int i){
        return accounts[i]; // not synchronized!
    }
}
```



Madhavan Mukund

Monitors in Java

Programming Concepts using Java

4 / 9

So, as before, we have a queue, an external queue, which is implicitly associated with that object. So, all the processes which are trying to get into a synchronized thing, but cannot are waiting in this external queue.

(Refer Slide Time: 7:04)



Monitors in Java

- `wait()` and `notify()` to suspend and resume
- Wait — single internal queue
- Notify
 - `notify()` signals one (arbitrary) waiting process
 - `notifyAll()` signals all waiting processes
 - Java uses **signal and continue**

```
public class bank_account{
    double accounts[100];

    public synchronized boolean
    transfer(double amount, int source, int target){
        while (accounts[source] < amount){ wait(); }
        accounts[source] -= amount;
        accounts[target] += amount;
        notifyAll();
        return true;
    }

    public synchronized double audit(){
        double balance = 0.0;
        for (int i = 0; i < 100; i++)
            balance += accounts[i];
        return balance;
    }

    public double current_balance(int i){
        return accounts[i]; // not synchronized!
    }
}
```



Madhavan Mukund

Monitors in Java

Programming Concepts using Java

5 / 9

Now, you also have, of course, this ability to exit and wait for the condition to be true. So, here we have this wait function. And correspondingly we have this notify all, so notify all is a curiosity. So, wait gives us a single internal q. So, for each object to remember in the earlier, more sophisticated bank account monitor, we said we will not have a single internal q where

we wait. But rather, we will create one internal queue for each bank account, because we know that a particular thread is waiting for a particular bank account to get some balance.

So, only when that account has the money transferred into it should I notify it. Well, Java does not have that at this point. It just has one implicit internal queue like that one implicit external queue, which is triggered by wait. Now, somewhat mysteriously, Java has two kinds of notifies. So, there is something which is just notify. And then there is something called notify all.

So, if you just say notify, then in some uncontrolled way that you as a programmer cannot control. Any one of the threads that is currently waiting, will get woken up. If you say notify all then all of them will get woken up. But of course, only one of them will be scheduled by the scheduler. And Java uses this more natural thing that we discussed signal and continue. So, when I say notify all, of course, here there is nothing more it is only return true, but if there had been more code than this transfer that is notified will complete before anything wakes up.

So, the thing is that we should typically use notify all because there is no point in waking up notify, which only guarantees only one thing will wake up because anyway notify all will wake up everyone but only one of them will get scheduled. So, you might as well do that.

(Refer Slide Time: 8:54)

IIT Madras
BSc Degree

Object locks ...

- Use object locks to synchronize arbitrary blocks of code

```
public class XYZ{  
    Object o = new Object();  
  
    public int f(){  
        ..  
        synchronized(o){ ... }  
    }  
  
    public double g(){  
        ..  
        synchronized(o){ ... }  
    }  
}
```

Madhavan Mukund

Monitors in Java

Programming Concepts using Java 6/9



IIT Madras
BSc Degree

Monitors in Java

- `wait()` and `notify()` to suspend and resume
- Wait — single internal queue
- Notify
 - `notify()` signals one (arbitrary) waiting process
 - `notifyAll()` signals all waiting processes
 - Java uses **signal and continue**

```

public class bank_account{
    double accounts[100];

    public synchronized boolean
    transfer(double amount, int source, int target){
        while (accounts[source] < amount){ wait(); }
        accounts[source] -= amount;
        accounts[target] += amount;
        notifyAll();
        return true;
    }

    public synchronized double audit(){
        double balance = 0.0;
        for (int i = 0; i < 100; i++)
            balance += accounts[i];
        return balance;
    }

    public double current_balance(int i){
        return accounts[i]; // not synchronized!
    }
}

```



Madhavan Mukund
Monitors in Java
Programming Concepts using Java 5/9

So, in addition to this notion, so here what we have is we have with every class that we define, we can associate synchronized methods, and those synchronized methods will be automatically mutually exclusive for every object that we create from that class. Now, separately, every object has a lock. So, whether it is being used as a method or not, we can just create an object and use that lock.

So, for instance, I can just say, I want a new object. This has no role to play except to provide me with a lock. And now there is a different form of synchronized, which says synchronize on the lock associated with that specific objective.

(Refer Slide Time: 9:42)



IIT Madras
BSc Degree

Object locks ...

- Use object locks to synchronize arbitrary blocks of code
- `f()` and `g()` can start in parallel
- Only one of the threads can grab the lock for `o`

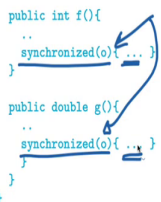
```

public class XYZ{
    Object o = new Object();

    public int f(){
        ..
        synchronized(o){ ... }
    }

    public double g(){
        ..
        synchronized(o){ ... }
    }
}

```





Madhavan Mukund
Monitors in Java
Programming Concepts using Java 6/9

So, I can have two functions now f and g here which both use this synchronized lock and supposing it was in the form of a thread now, the functions themselves are not synchronized. So, there is no synchronized here, which indicates that these functions are mutually exclusive. So, f and g can start in parallel through different threads. But this synchronized lock that we have asked for here can only go to one of them. So, only one of them can get past this requirement of grabbing that lock.

So, the other one will be asked to wait here. So, this is a more fine grained control that we have in this then by making the entire method synchronized. So, we are saying there is a particular part. So, this is remember when we talked about critical sections, so there is some kind of code here, which manipulates the state of some object that we are interested in. And we are trying to make sure that these two functions f and g do not do that simultaneously to avoid any corruption of the data.

(Refer Slide Time: 10:44)

Object locks ...

- Use object locks to synchronize arbitrary blocks of code
- `f()` and `g()` can start in parallel
- Only one of the threads can grab the lock for `o`
- Each object has its own internal queue

```
Object o = new Object();

public int f(){
    ..
    synchronized(o){
        ...
        o.wait(); // Wait in queue attached to "o"
        ...
    }
}

public double g(){
    ..
    synchronized(o){
        ...
        o.notifyAll(); // Wake up queue attached to "o"
        ...
    }
}
```

Madhavan Mukund | Monitors in Java | Programming Concepts using Java | 6 / 9

So, just like when we defined a normal class, and we said that when we have a synchronized method on that class, then anything that cannot get into the synchronous method is implicitly waiting in the queue associated with that object. Similarly, here, when I am using an external object o, it also has this queue, which is there for synchronized, but it also has an internal queue. So, I can say that, I want to give up o, because I am waiting for something useful to happen. And I want to be notified.

So, just like you can do a wait inside a normal synchronized method and be notified within that implicitly. Here, I can explicitly say I am waiting on o, and some other thread can then notify everybody who is waiting all over. So, each object has both an external queue which I can trigger using this synchronized function. And an internal queue, which I can call in the context of that object. So, instead of just saying wait, I say o dot wait. So, I am waiting for something in the context of o.

(Refer Slide Time: 11:50)

Object locks ...

- Use object locks to synchronize arbitrary blocks of code
- `f()` and `g()` can start in parallel
- Only one of the threads can grab the lock for `o`
- Each object has its own internal queue
- Can convert methods from "externally" synchronized to "internally" synchronized

```
public double h(){  
    synchronized(this){  
        ...  
    }  
}
```

Madhavan Mukund | Monitors in Java | Programming Concepts using Java | 6 / 9

Now, interestingly, you can also do this within an object within, within the class itself. You do not have to necessarily go outside and manufacture new thing you can say I want to synchronize on this object. So, the function again, is not synchronized. So, this is just a normal function. So, double can be started by more than one or double can be started in parallel with something else which is not synchronized, it will work. But when we get to this point, only one function will be, one of these copies of these threads, which is trying to execute h will be able to grab that lock associated with this object.

And in particular, this lock is the same as the lock that goes with the synchronized in the function. So, this will kind of me in competition with everything that is synchronized on this object. So, this is the kind of, instead of using an external object o to synchronize a block of code, we can even use the current object internally to synchronize a block of code.

(Refer Slide Time: 12:46)



IIT Madras
BSc Degree

Object locks ...

- Use object locks to synchronize arbitrary blocks of code
- `f()` and `g()` can start in parallel
- Only one of the threads can grab the lock for `o`
- Each object has its own internal queue
- Can convert methods from "externally" synchronized to "internally" synchronized
- "Anonymous" `wait()`, `notify()`, `notifyAll()` abbreviate `this.wait()`, `this.notify()`, `this.notifyAll()`

```
public double h(){
    synchronized(this){
        ...
    }
}
```



Madhavan Mukund

Monitors in Java

Programming Concepts using Java 6 / 9

So, really, when we were saying earlier wait and notify and notify all without saying anything, it was actually just a special case of this `o dot wait`, `o dot notify`, `o dot notify all` we are actually saying `this dot wait`, `this dot notify`. I am waiting on the internal queue for this object for the current myself and waiting on or I am notifying everybody who is waiting on the internal queue for myself. So, this is all actually the same thing, but used in different ways.

(Refer Slide Time: 13:12)



IIT Madras
BSc Degree

Object locks ...

- Actually, `wait()` can be "interrupted" by an `InterruptedException`
- Should write

```
try{
    wait();
}
catch (InterruptedException e) {
    ...
};
```

Busy wait

while (request != true){}



Madhavan Mukund

Monitors in Java

Programming Concepts using Java 7 / 9

So, in many of these things, we are a little careful about exceptions and wait is one of those. So, wait can actually be interrupted. So, technically speaking, you should always put wait in a

try and catch an interrupted exception. And we will see a little later why we need to do this. So, an interrupted exception basically comes when this thread gets a signal from another thread. So you can get a signal and then you have to react to that signal, even though you are currently inactive. So, this is the problem.

If you are active, then you could be checking for the signal. But now by putting yourself in a wait state. So, when a Java thread goes into a wait state or into a blocked state, so, remember, we talked earlier about something called a busy wait. So, when we did our initial mutual exclusion thing we said while request 1 not equal to true, do something. So, this was actively checking, in some sense whether this value is true or false. Now when we go into this busy in this q, external or internal, either we try to access synchronous method or we wait.

The process itself is asleep. It is not taking any resources, the scheduler is waiting to know whether these things are runnable again, whether they can bring them back to life. So, then it will get woken up. So, in this situation, this process is really interrupted. It is not monitoring anything, it has no control over its variables. So, now at this point, if you want to tell that process something by sending it what is called an interrupt the signal, then the only way you can get it is by getting an exception.

So, that is why many of these things where earlier we saw that and we will see again that if you go to sleep for instance, and you are waiting for something to happen, supposing it says sleep for 1 second, then during that 1 second, if anything happens again, you are in an inert state, so you must be ready to receive the signals whether you are active or not. So, that is why this interrupt exception keeps coming wherever there is a situation, where the process is voluntarily in an inert state.

(Refer Slide Time: 15:17)



Object locks ...

- Actually, `wait()` can be "interrupted" by an `InterruptedException`
- Should write

```
try{
    wait();
}
catch (InterruptedException e) {
    ...
};
```
- Error to use `wait()`, `notify()`, `notifyAll()` outside synchronized method
 - `IllegalMonitorStateException`

7/5
✓5



Madhavan Mukund

Monitors in Java

Programming Concepts using Java

7/9

So, of course, we saw when we looked at semaphores, remember, in a semaphore, you had these 2 operations P of s and V of s and P of s in some sense, grab the semaphore by decrementing a counter and V of s release that semaphore by incrementing a counter. And we said that, in a semaphore, the problem is that these things are supposed to be matched P supposed to be followed by V, but they could be due to some quirk of your coding, you execute a V before a P and then you have the whole semaphore in some unknown state, because you have released a semaphore which was already released.

And then what do you do after that, because mutual exclusion will not be guaranteed, because now the counter which used to be 1 has gone to 2, so, 2 decrements are possible, and that will violate mutual exclusion. So, in the same way, you could argue that in this notify setting or wait setting, I mean, even if you are not in a situation where you are in a position to wait, you asked to wait or you ask notify, then what happens? So, Java does not allow this.

So, if you are not in a synchronized method, then wait is disallowed. If you are not in a synchronized method, notify is disallowed. So, you even though every object has this queue, so, the queue is there, always. But if you are not in a valid state to use that queue, by waiting in it for getting notified or telling the q that you can wake up things, then it is illegal, so you get an illegal monitor state exception. So, here you can see in this the word monitor coming. So, Java acknowledges that it is actually using this monitor construct in order to achieve the synchronization.

(Refer Slide Time: 16:50)



Object locks ...

- Actually, `wait()` can be "interrupted" by an `InterruptedException`
- Should write

```
try{
    wait();
}
catch (InterruptedException e) {
    ...
};
```
- Error to use `wait()`, `notify()`, `notifyAll()` outside synchronized method
 - `IllegalMonitorStateException`
- Likewise, use `o.wait()`, `o.notify()`, `o.notifyAll()` only in block synchronized on `o`



Madhavan Mukund

Monitors in Java

Programming Concepts using Java

7/9

So, similarly, if I ask to wait on an object `o`, or to notify everybody who is waiting in a queue for `o`, unless I am in a block, which is synchronized with `o`, unless I have a legitimate reason to do so, it gives me this kind of monitor state exception. So, you cannot do wait or notify unless you are in the correct context to do that.

(Refer Slide Time: 17:14)



Reentrant locks

- Separate `ReentrantLock` class
- Similar to a semaphore
 - `lock()` is like P(S)
 - `unlock()` is like V(S)

```
public class Bank {
    private Lock bankLock = new ReentrantLock();
    ...
    public void
    transfer(int from, int to, int amount) {
        bankLock.lock();
        try {
            accounts[from] -= amount;
            accounts[to] += amount;
        }
        finally {
            bankLock.unlock();
        }
    }
}
```



Madhavan Mukund

Monitors in Java

Programming Concepts using Java

8/9

So, this is the notion of a monitor. So, Java has basically taken the notion of a monitor and kind of integrated it smoothly, because it is natural, because monitors are very much like classes, it is very, it is a very smooth integration to convert an arbitrary Java class into a

monitor by just adding this. So, unlike in a monitor, remember in a monitor everything is what Java would call a synchronous method.

So, you have a data definition and every function in a monitor is technically synchronized. Whereas in Java, you have a data definition and then you can specify which functions are synchronized. So, you can have some which are synchronized and some paths which are not dangerous, and you can allow them to be run in parallel. But in addition to this later on, Java has also introduced locks. And locks are pretty much a kind of modern version of semaphores.

So, remember, in the semaphore, you grabbed it was like a counter, and you grab the semaphore by saying P which subtracted 1 if it was positive, and if it is not positive, you wait. And when you say V, you add 1 and therefore somebody who is waiting for it to become positive is now able to complete their P of s. So, in the lock context, these things are explicitly called lock and unlock.

So, grabbing a P is like grabbing hold of the lock. So, that is usually a function called lock and taking it giving it back is unlocked. So, here is a way you do that. So, you create so there is a data type class in Java now called lock it is there in one of the in the Java dot util dot concurrent. So, you have a class called lock, which has different types of lock and in particular, it has this thing called a ReentrantLock.

So, you create a variable of this type an object of type ReentrantLock and now I want to do this transfer. So, this transfer is protected by this particular lock. So, any process that wants a new thread that wants to do transfer has to first lock this bank lock. And if one processes already using bank lock, when this bank lock is not available, so it has to wait. So, they will be again a queue associated.

(Refer Slide Time: 19:31)



Reentrant locks

- Separate `ReentrantLock` class
- Similar to a semaphore
 - `lock()` is like P(S)
 - `unlock()` is like V(S)
- Always `unlock()` in `finally` — avoid abort while holding lock

```
public class Bank
{
    private Lock bankLock = new ReentrantLock();
    ...
    public void
    transfer(int from, int to, int amount) {
        bankLock.lock();
        try {
            accounts[from] -= amount;
            accounts[to] += amount;
        }
        finally {
            bankLock.unlock(); // V(S)
        }
    }
}
```



Madhavan Mukund

Motivations in Java

Programming Concepts using Java 8 / 9

And now, what we will do is we will finish this and then finally we will unlock. Now, you would normally just do this if it is a semaphore. So, this is like p of s, do your code and then do a V of s. But the only thing is that we want to make sure that there is no situation where I have done P of s grab the lock. And then for whatever reason this code here through an exception and then the rest of the code did not execute. So, the V of s never happened. So, you remember that if a thread has got a lock, and it crashes for whatever reason, then the lock is forever stuck with that thread so nobody else can actually get it.

So, the recommended thing, therefore, in Java is to always take after you grab the lock, whatever you want to do, you put it into a try. And you are not putting it into a try necessarily, because you want to catch an exception, but because you want to make sure that the unlock always happens. So, remember this finally block in a try thing is something which executes whether the try exits normally or the try exits with an exception, so Java guarantees that the final code is going to be executed.

So, you put your unlock into the finally code to make sure that you never die while holding the lock.

(Refer Slide Time: 20:47)



IIT Madras
BSc Degree

Reentrant locks

- Separate `ReentrantLock` class
- Similar to a semaphore
 - `lock()` is like `P(S)`
 - `unlock()` is like `V(S)`
- Always `unlock()` in `finally` — avoid abort while holding lock
- Why `reentrant`?
 - Thread holding lock can reacquire it
 - `transfer()` may call `getBalance()` that also locks `bankLock`
 - Hold count increases with `lock()`, decreases with `unlock()`
 - Lock is available if hold count is 0

```
public class Bank
{
    private Lock bankLock = new ReentrantLock();
    ...
    public void
    transfer(int from, int to, int amount) {
        bankLock.lock();
        try {
            accounts[from] -= amount;
            accounts[to] += amount;
        }
        finally {
            bankLock.unlock();
        }
    }
}
```



Madhavan Mukund

Motivations in Java

Programming Concepts using Java

8 / 9

So, why is this Reentrant? The reason that it is `ReentrantLock` is that, supposing I have 2 functions that require locking supposing I have this transfer. And this transfer also requires me to, for example, maybe get the bank balance as it is, as I would expect through a different function. But maybe get bank balances also locked using this lock, because again, that you do not want inconsistent balances to be reported. So, it is a little bit like the wait situation, but it is slightly different.

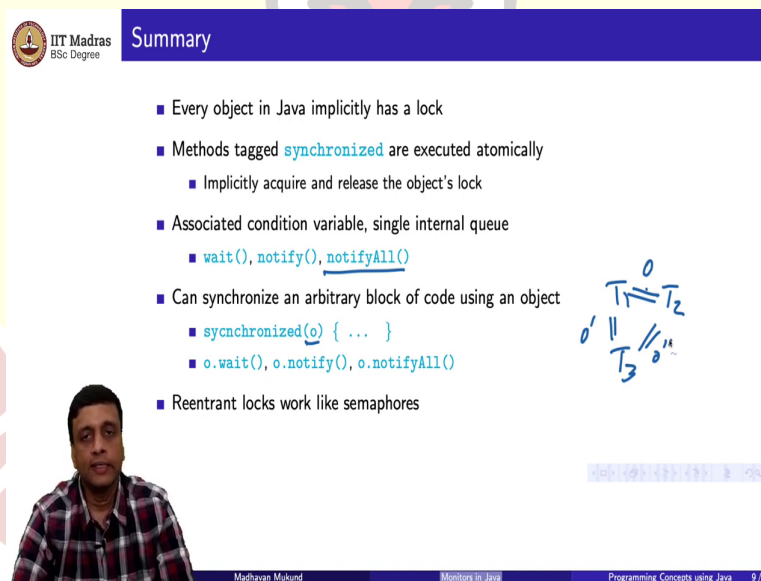
So, here I have a thread, which already has the lock that it needs. And now it comes to this next function, which asks to have the same lock. So, rather than checking if I have a lock and doing something complicated, what these reentrant locks do is they just say, if you have the lock, then you can take it again. So, you can re-enter the lock. So, each time you re-enter the lock, you kind of add implicitly to a counter of how many times you hold the lock. So, each time you reenter it through a different function, assuming you can only get it if you already have it.

So, another process, which does not have the lock cannot enter a reentrant lock. But you as the thread which has the lock can reenter the reentrant lock. So, each time you do a lock, assuming it succeeds, you increment the counter, which is there. So, you can do this with semaphores also, sometimes it is called the counting semaphore, but there you are counting down. So, then you have to decide in advance how much you can count down. Whereas this is clever, he starts with 0, and it counts up so you can go as far as you want.

But then only you can release these codes. So, every time you do an unlock, it comes back down. So, you lock once, you unlock once it comes back to 0, somebody else can take it. You unlock twice, then you must unlock twice, so that somebody else can take it. So, this is the reentrant lock. But still, this is more manual and more like semaphores, then the monitor case. In the monitor case, it is more abstract, you are saying here is a function that I want to define, which I do not want interference with so executed atomically. Whereas, here we are saying here is a block of code.

And I am going to promise you before I get in that I lock it and after I get that out that I unlock it. And then I have to remember to put it in finally and all that. So, this is definitely more fine grained, but also more error prone version, which is very similar to semaphores. But it is available in Java. So, you can explicitly use locks that is something like a semaphore. Or you can use the monitor thing through synchronized. And then in that you can also synchronize on an arbitrary object. So, you can just take a block of code and synchronize it.

(Refer Slide Time: 23:21)



Summary

- Every object in Java implicitly has a lock
- Methods tagged `synchronized` are executed atomically
 - Implicitly acquire and release the object's lock
- Associated condition variable, single internal queue
 - `wait()`, `notify()`, `notifyAll()`
- Can synchronize an arbitrary block of code using an object
 - `synchronized(o) { ... }`
 - `o.wait()`, `o.notify()`, `o.notifyAll()`
- Reentrant locks work like semaphores

Handwritten notes on the right side of the slide:

```

0
T1 = T2
o' || T3
    
```

Video inset shows a man speaking.

Footer: Madhavan Mukund | Monitors in Java | Programming Concepts using Java | 9/9

So, to summarize, every object in Java implicitly has a lock. So, using this lock, you can tag methods are synchronized and guarantee that these synchronized methods never interfere with each other. They will all be executed atomically from beginning to end, without any threat interleaving in between. And associated with this objects lock, you have a single condition variable where you can wait.

So, you can wait, you can notify, you can notifyAll and as we said normally you would normally as a programmer, you will do notifyAll. But you can also take an arbitrary object and synchronize based on that not on the current object. So, you can take some object, any object o, every object in Java has got a lock. So, you can say I want to synchronize on that object. So, you can say these 2 threads synchronize on one object, this thread, so you can have a very diverse thing.

So, you could have T1, T2 and T3. And these 2 could synchronize on some o, and these 2 can synchronize on some o prime. And these 2 can synchronize on some o double prime. So, they can all have a different convention as to how they so they are not all tied to synchronizing on the same thing. So, when you have the synchronized objects, we can do a lot of flexible programming, but of course, the more flexible your programming is, the more error prone it is. So, you have to be little careful, and then finally, like semaphores Java allows you these reentrant locks.

